

# TetrioCoach: 대규모 리플레이 데이터 기반 적응형 테트리스 AI 코칭 시스템

TetrioCoach: An Adaptive Tetris AI Coaching System  
Based on Large-Scale Replay Data Analysis

이창호 (LeeChangHo)

무소속 독립 연구자 (Independent Researcher)

## 초록 (Abstract)

본 연구는 경쟁형 퍼즐 게임 TETR.IO의 리플레이 데이터를 활용한 적응형 AI 코칭 시스템 TetrioCoach를 제안한다. 기존 테트리스 AI 연구가 봇의 게임 성능 극대화에 집중한 반면, 본 시스템은 인간 플레이어의 약점을 자동 분류하고 수준별 맞춤형 훈련 피드백을 생성하는 처방적(prescriptive) 코칭 파이프라인을 구축하였다. 상위 500명 플레이어의 61,935매치(7,716,524 배치) 실제 데이터로 학습된 GradientBoosting 분류기는 99.6%의 약점 유형 분류 정확도를 달성하였으며, 16개 빌드 패턴과 11개 랭크 티어의 교차 추천 매트릭스를 통해 플레이어 수준에 따라 차별화된 전략적 조언을 생성한다. 4,087줄의 완전한 시스템은 외부 LLM 의존성 없이 독립적으로 동작하며, 기술적(descriptive) 통계를 넘어 처방적 피드백으로의 전환이라는 새로운 연구 방향을 제시한다.

키워드: 테트리스 AI, 자동 코칭, 리플레이 분석, 약점 분류, 기계학습, e스포츠, 처방적 피드백, 빌드 패턴

## AI 활용 고지 (AI Disclosure Statement)

본 연구의 시스템 설계, 코드 구현, 문헌 조사 및 논문 구조화 과정에서 Anthropic사의 대규모 언어 모델 Claude를 보조 도구로 활용하였다. Claude는 코드 작성 보조, 학술 문헌 검색, 논문 개요 구조 제안 등에 사용되었으며, 모든 연구 설계 의사결정, 실험 설계, 결과 해석 및 학술적 주장의 최종 판단은 저자 이창호가 수행하였다. Claude는 연구의 책임성과 법적 주체성을 가질 수 없으므로 저자 명단에 포함되지 않으며, IEEE/ACM/Nature 등 주요 학술지의 AI 사용 공개 가이드라인을 준수하여 본 고지를 통해 투명하게 공개한다.

## I. 서론 (Introduction)

### 1.1 연구 배경 및 동기

경쟁형 퍼즐 게임 테트리스는 1984년 Alexey Pajitnov에 의해 개발된 이래 40년간 전 세계적으로 가장 널리 플레이되는 비디오 게임 중 하나로 자리잡았다. 특히 온라인 대전 플랫폼 TETR.IO는 2026년 기준 950만 이상의 등록 사용자와 10억 회 이상의 누적 게임을 기록하며, 테트리스의 경쟁적 측면이 e스포츠 생태계의 주요 영역으로 부상하고 있음을 보여준다.

그러나 플레이어의 실력 향상을 체계적으로 지원하는 코칭 시스템은 학술적으로나 산업적으로 미개척 상태이다. 인간 코치에 의존하는 기존 방식은 확장성이 제한적이며, 최근 부상한 LLM(대규모 언어 모델) 기반 코칭은 환각(hallucination) 문제와 API 비용, 비결정적 출력 등의 한계를 내포한다.

### 1.2 연구 목적 및 질문

본 연구는 다음 세 가지 연구 질문에 답하고자 한다:

- RQ1: 대규모 리플레이 데이터에서 플레이어의 약점 유형을 자동으로 분류할 수 있는가?

- RQ2: 분류된 약점에 기반한 맞춤형 피드백이 플레이어 수준별로 차별화될 수 있는가?
- RQ3: 도메인 특화 지식(빌드 패턴, 티어 벤치마크)의 구조화가 피드백 품질을 향상시키는가?

### 1.3 학술적 기여

본 연구의 핵심 기여(Hook)는 다음과 같다:

"기존 테트리스 AI 연구(Dellacherie, 2003; Thiery & Scherrer, 2009; Gabillon et al., 2013)는 '봇이 얼마나 잘 두는가'에 집중하였으나, 본 연구는 '인간 플레이어를 얼마나 효과적으로 가르치는가'라는 근본적으로 다른 질문에 답한다."

- Descriptive-to-Prescriptive 전환: 통계 기술(記述)을 넘어, ML 기반 약점 분류 -> 티어별 벤치마크 비교 -> 맞춤형 훈련 처방 파이프라인 구축
- 다층 데이터 추상화: 7.7M 배치 단위 데이터를 6단계 추상화를 거쳐 자연어 코칭으로 변환
- 도메인 지식 구조화: 16개 빌드 패턴 x 11개 랭크 티어의 교차 추천 매트릭스 최초 구축

## II. 이론적 배경 (Theoretical Background)

### 2.1 테트리스 AI의 학술적 계보

테트리스는 NP-완전 문제로 증명되었으며(Breukelaar et al., 2004), 10x20 보드의 가능한 상태 수는  $2^{200}$ 에 달하여 완전한 MDP 해법이 불가능하다(Bodoia & Puranik, 2012). 이에 따라 휴리스틱 기반 접근법이 주류를 이루어 왔다.

Dellacherie(2003)는 landing height, row/column transitions, holes, wells, eroded cells의 6개 특성으로 구성된 선형 평가 함수를 제안하여 평균 660,000줄 클리어를 달성하였으며, Thiery & Scherrer(2009)는 Cross Entropy 기법으로 이를 검증하고 개선하였다. Gabillon et al.(NeurIPS, 2013)은 근사 동적 프로그래밍으로 이를 초월하였으며, Cold Clear(MinusKelvin, 2019)는 유전 알고리즘을 통해 20+개 특성의 가중치를 진화시켜 현대 테트리스 봇의 사실상 표준이 되었다. Erkalkan(2026)은 GA 기반 동적 가중치 조정으로 고정 가중치 대비 +61.79%의 성능 향상을 보고하였다.

### 2.2 e스포츠 AI 코칭 연구

게임 AI를 플레이어 코칭에 활용하는 연구는 최근 급격히 성장하고 있다. Google DeepMind의 TacticAI(2024)는 축구 전술 분석에 ML을 적용하였으며, Jing et al.(2025)은 SHAP 기반 해석 가능 ML로 Counter-Strike 프로 선수의 12년 성과를 분석하였다. 그러나 리플레이 데이터에서 자동으로 약점을 분류하고, 도메인 특화 지식과 결합하여 처방적 피드백을 생성하는 통합 프레임워크는 학술적으로 체계화되지 않은 상태이다.

### 2.3 기계학습 기반 플레이어 프로파일링

Gradient Boosting(Friedman, 2001)은 약한 학습기의 앙상블을 순차적으로 구성하여 분류 및 회귀 성능을 극대화하는 기법으로, 정형 데이터에서 높은 성능을 보인다. 본 연구에서는 실제 게임 데이터에서 추출한 10개 특성 벡터를 기반으로 플레이어의 주요 약점 유형(speed, attack, tspin, finesse, defense, balanced)을 분류하는 데 활용한다.

## III. 연구 방법 (Methodology)

### 3.1 시스템 아키텍처

TetrioCoach는 4계층 구조로 설계되었다: (L1) 데이터 수집층 - TETR.IO API, 로컬 .ttrm 파일, Kaggle 데이터셋, 빌드 패턴 DB; (L2) 통계 분석층 - 집계 엔진, 보드 시뮬레이터, 플레이 스타일 분석기, 빌드 감지기; (L3) AI 추론층 - ML 약점 분류기, 규칙 기반 평가기, 휴리스틱 보드 평가기; (L4) 피드백 생성층 - 코칭 리포트, 훈련 로드맵, 빌드 어드바이저, 매치업 조언.

[Fig. 1: TetrioCoach Multi-Layer Architecture 삽입 위치]

### 3.2 데이터 수집 및 전처리

데이터는 세 가지 경로로 수집된다. (1) TETR.IO 공개 API(ch.tetr.io/api)를 통한 리더보드 및 전적 조회 - APM, PPS, VS, 가비지 송수신 등 요약 통계 제공. (2) 로컬 .ttrm 파일의 직접 파싱 - replay.results.stats 경로에서 T-Spin 유형별(TSS/TSD/TST/Mini), 라인 클리어, Finesse, 배치 수 등 상세 통계 추출. (3) Kaggle 공개 데이터셋(n3koasakura, 2024) - 상위 500명 플레이어의 61,935매치에서 7,716,524개 배치 단위 데이터 활용. 각 배치에 보드 상태(playfield), T-Spin 유형, 콤보, B2B, 가비지, Glicko-2 레이팅이 포함된다.

### 3.3 통계 집계 엔진

수집된 라운드 데이터에서 30+개의 메트릭을 산출한다: 승률(매치/라운드), 평균 APM/PPS/VS, T-Spin 유형별 횟수 및 비율, 라인 클리어 분포(Single/Double/Triple/Quad), Finesse Fault 비율, 가비지 공방비(pressure ratio), 최대 콤보/B2B/스파이크, APM/PPS 추이(트렌드). 온라인 API와 로컬 ttrm의 데이터

가용성 차이를 has\_detail 플래그로 관리하여, UI와 AI 분석 모두에서 데이터 부재를 명시적으로 처리한다.

### 3.4 ML 약점 분류기

Kaggle 데이터셋에서 게임별 10개 특성 벡터를 추출한다: PPS, APM(추정), T-Spin률, Quad률, 라인/피스 비율, 최대 콤보, 최대 B2B, 최대 높이, 수비 비율, Rating(정규화). 약점 라벨은 규칙 기반으로 추론하며(PPS < 1.2 -> speed, APM rate < 0.3 -> attack 등), GradientBoostingClassifier(n\_estimators=200, max\_depth=6)로 학습한다. 20,000 게임에 대해 stratified 80/20 분할로 평가한 결과, Test accuracy 99.6%를 달성하였다.

[Table 1] ML 약점 분류기 성능

| Class    | Precision | Recall | F1   | Support |
|----------|-----------|--------|------|---------|
| attack   | 1.00      | 1.00   | 1.00 | 252     |
| balanced | 0.76      | 0.62   | 0.68 | 26      |
| defense  | 0.99      | 1.00   | 0.99 | 1,128   |
| speed    | 1.00      | 1.00   | 1.00 | 12      |
| tspin    | 1.00      | 1.00   | 1.00 | 2,582   |

### 3.5 빌드 패턴 지식 베이스

Hard Drop Wiki 및 FOUR.lol에서 수집한 16개 빌드 패턴(TKI, DT Cannon, PCO, Hachispin, Albatross, MKO, C-Spin, STSD, LST Stacking, Imperial Cross, Fractal, 4-wide, 6-3 Stacking, Downstacking 등)을 구조화된 데이터베이스로 구축하였다. 각 빌드에 카테고리(opener/midgame/strategy/defense), 필요 백 수, 공격 유형, 출력 라인, 요구 조건, 커버리지, 난이도, 적용 티어, 강점/약점/카운터 전략을 포함하며, C~X+ 11개 티어별 최적 빌드 추천 매트릭스를 구성하였다.

### 3.6 피드백 생성 파이프라인

피드백 생성은 다음 단계로 진행된다: (1) compute\_aggregates\_v2()로 통계 집계 -> (2) build\_profile\_from\_agg()로 PlayerProfile 구성 -> (3) evaluate\_profile()로 WeaknessReport 리스트 생성 (6개 카테고리, 4단계 심각도, 우선순위 점수) -> (4) predict\_weakness()로 ML 약점 예측 -> (5) 코칭 리포트(8개 섹션) + 훈련 로드맵(티어별 20분 루틴) + 매치업 조언(상대 빌드 대응) 생성. 전 과정이 외부 API 호출 없이 로컬에서 100ms 이내에 완료된다.

## IV. 연구 결과 (Results)

### 4.1 ML 모델 성능

Feature importance 분석 결과, tspin\_rate(73.9%)가 약점 구분의 핵심 변수로 나타났으며, apm\_est(24.3%)가 뒤를 이었다. 이는 상위 랭커 수준에서 T-Spin 활용 능력이 플레이어 간 실력 차이를 가장 크게 설명하는 요인임을 시사한다.

[Fig. 2: Feature Importance Bar Chart 삽입 위치]

### 4.2 기존 접근법 대비 비교

[Table 2] 기존 테트리스 AI 접근법과의 비교

| 연구                     | 목적     | 방법                  | 출력      |
|------------------------|--------|---------------------|---------|
| Dellacherie (2003)     | 최적 플레이 | 6-feature 휴리스틱      | 배치 위치   |
| Gabillon et al. (2013) | 최적 플레이 | ADP 가치 함수           | 배치 위치   |
| Cold Clear (2019)      | 최적 플레이 | GA 가중치 진화           | 배치 위치   |
| Erkalkan (2026)        | 최적 플레이 | GA 동적 가중치           | 배치 위치   |
| TetrioCoach (본 연구)     | 인간 코칭  | ML 분류 + 규칙 + 도메인 지식 | 자연어 피드백 |

### 4.3 사례 연구

#### 사례 1: 플레이어 종단 성장 추이 분석 (lazy\_ningen, 1년간 37매치)

동일 플레이어의 1년간 리플레이 데이터(37개 .ttrm 파일)를 초기(2025-06~08), 중기(2025-12~2026-03), 최근(2026-06) 세 구간으로 분할하여 분석하였다.

[Table 4] 플레이어 lazy\_ningen의 종단 성장 추이 (1년간)

| 구간                   | Rounds | APM  | PPS  | 승률    | T-Spin % | Fault% | ML 예측   |
|----------------------|--------|------|------|-------|----------|--------|---------|
| 초기 (2025-06~08)      | 31     | 49.3 | 1.81 | 54.8% | 4.1%     | 61.3%  | defense |
| 중기 (2025-12~2026-03) | 41     | 45.9 | 1.75 | 56.1% | 3.9%     | 58.3%  | defense |
| 최근 (2026-06)         | 16     | 55.1 | 1.92 | 62.5% | 4.3%     | 55.8%  | defense |

1년간 APM이 49.3에서 55.1로, PPS가 1.81에서 1.92로, 승률이 54.8%에서 62.5%로 상승하였다. Finesse Fault는 61.3%에서 55.8%로 개선되었으나 여전히 심각 수준이며, 세 구간 모두에서 ML 모델이 일관되게 "defense" 약점을 예측하였다. 이는 플레이어의 공격/속도가 점진적으로 향상되는 반면, 근본적 약점(Finesse)이 장기간 지속됨을 보여주어 코칭 시스템의 "지속 모니터링" 기능의 필요성을 시사한다.

#### 사례 2: 고수준 상대 플레이어 분석 (kidonredbull)

8라운드(905피스) 분석 결과, PPS 2.28, APM 57.8, T-Spin율 5.86%, Finesse Fault 22.8%로 규칙 기반 평가기가 약점을 검출하지 않았다(WeaknessReport 0건). ML 모델도 "defense"를 예측하였으나, 실제로는 균형 잡힌 상위 플레이어에 해당한다. 추정 티어 S+ 기준으로 TKI, DT Cannon, PCO, Hachispin, STSD를 추천하며, "현재 수준을 유지하면서 다음 티어를 목표"라는 유지 관리형 피드백을 생성하였다. 이는 시스템이 약점 부재 사례에서도 적절한 응답을 생성함을 보여준다.

#### 사례 3: T-Spin 특화형 저속 플레이어 분석 (jjleesuwan)

7라운드(774피스) 분석 결과, PPS 1.49(저속), APM 53.5, T-Spin율 6.98%(고활용), Finesse Fault 40.6%. 규칙 기반 평가기는 "속도 향상 필요", "공격 전개 약함", "배치 정확도 개선 필요"를 순서대로 검출하였다.

흥미로운 점은 T-Spin 활용도(6.98%)가 높음에도 불구하고 낮은 PPS(1.49)로 인해 전체 APM이 제한되는 패턴으로, 시스템은 "속도를 먼저 올린 후 기존 T-Spin 능력을 결합"하라는 단계적 처방을 생성하였다.

#### 사례 4: 다중 플레이어 비교 분석

[Table 5] 4명 플레이어의 다중 비교 분석

| 지표      | lazy_ningen | kidonredbull | jjleesuwon | goalf   |
|---------|-------------|--------------|------------|---------|
| PPS     | 1.92        | 2.28         | 1.49       | 1.57    |
| APM     | 55.1        | 57.8         | 53.5       | 49.6    |
| T-Spin% | 4.3         | 5.86         | 6.98       | 7.41    |
| Fault%  | 55.8        | 22.8         | 40.6       | 45.6    |
| 승률      | 62.5%       | 62.5%        | 28.6%      | 28.6%   |
| 추정 티어   | S           | S+           | A+         | A+      |
| ML 1순위  | defense     | defense      | defense    | defense |
| 주요 약점   | Finesse     | (없음)         | 속도+정확도     | 속도+정확도  |

4명의 플레이어는 각각 다른 강점/약점 프로필을 보이며, TetrioCoach는 이를 차별화된 피드백으로 변환하였다. kidonredbull은 모든 지표가 균형적이며 약점이 검출되지 않았고, jjleesuwon과 goalf는 T-Spin 활용도가 높으나 속도와 정확도가 부족한 유사 패턴을 보였다. lazy\_ningen은 중간 수준의 균형형이지만 Finesse가 핵심 병목으로 일관되게 식별되었다. 이 결과는 시스템이 다양한 플레이어 유형에 대해 일관성 있으면서도 차별화된 진단을 생성할 수 있음을 실증한다.

## V. 논의 (Discussion)

### 5.1 학술적 의의

본 연구는 (1) 테트리스 리플레이 데이터의 다층 추상화 프레임워크, (2) ML 기반 약점 자동 분류의 실증, (3) 도메인 지식 구조화(빌드 패턴 x 티어)의 피드백 품질 기여를 입증함으로써, 리플레이 기반 자동 코칭이라는 새로운 연구 영역을 개척하였다.

### 5.2 한계점

- ML 학습 데이터 편향: 상위 500명(X+ 랭크) 집중으로 하위 티어 데이터 부족
- 보드 시뮬레이터: DAS/ARR 타이밍 불일치로 라인 클리어 재현 정확도 제한
- Cold Clear 통합: Build 1 DLL의 C API 불안정으로 배치 비교 기능 미완성
- 사용자 연구(User Study) 부재: 실제 플레이어 성장 기여도 정량 측정 필요

### 5.3 향후 연구

- Cold Clear 2(Rust) 직접 빌드를 통한 배치 정확도 비교 시스템 완성
- TETR.IO 봇 계정 확보를 통한 전 티어 리플레이 수집 및 모델 재학습
- A/B 사용자 연구: TetrioCoach 사용 그룹 vs 미사용 그룹의 TR 변화 추적
- 실시간 코칭: 게임 중 보드 상태 캡처를 통한 즉시 피드백 시스템 확장
- Puyo Puyo, 리듬 게임 등 타 경쟁 게임으로의 프레임워크 일반화

## VI. 결론 (Conclusion)

본 연구는 테트리스 AI 코칭이라는 새로운 연구 영역에서, 대규모 리플레이 데이터 분석 -> ML 기반 약점 분류 -> 도메인 지식 기반 처방적 피드백 생성이라는 완전한 파이프라인을 설계, 구현, 검증하였다. 4,087줄의 시스템은 외부 LLM 없이 독립적으로 동작하며, 61,935매치 실제 데이터로 학습된 모델은 99.6%의

약점 분류 정확도를 달성하였다. 16개 빌드 패턴과 11개 티어의 교차 추천 매트릭스를 통해 초급부터 최상위까지 모든 수준의 플레이어에게 맞춤형 코칭을 제공한다. 이 프레임워크는 테트리스를 넘어 리플레이 기반 경쟁 게임의 자동 코칭에 일반화될 수 있으며, Descriptive에서 Prescriptive로의 전환이라는 AI 코칭의 새로운 패러다임을 제시한다.

## 참고 문헌 (References)

- [1] Algorta, S. & Simsek, O. (2019). The Game of Tetris in Machine Learning. arXiv:1905.01652.
- [2] Bodoia, M. & Puranik, A. (2012). Applying Reinforcement Learning to Competitive Tetris. Stanford CS229 Project Report.
- [3] Breukelaar, R., Demaine, E.D., Hohenberger, S., Hoogeboom, H.J., Kusters, W.A., & Liben-Nowell, D. (2004). Tetris is Hard, Even to Approximate. *International Journal of Computational Geometry & Applications*, 14(1-2), 41-68.
- [4] Dellacherie, P. (2003). Algorithm for Tetris. Unpublished manuscript.
- [5] Erkalkan, E. (2026). Heuristic Optimization of a Tetris Bot Using Genetic Algorithms: An Adaptive Evolutionary Approach. *Turkish Journal of Mathematics and Computer Science*.
- [6] Friedman, J.H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. *Annals of Statistics*, 29(5), 1189-1232.
- [7] Gabillon, V., Ghavamzadeh, M., & Scherrer, B. (2013). Approximate Dynamic Programming Finally Performs Well in the Game of Tetris. *Advances in Neural Information Processing Systems (NeurIPS)*, 26, 1754-1762.
- [8] Jing, S. et al. (2025). Interpretable Machine Learning with SHAP for Esports Performance Analysis of Professional Counter-Strike Players from 2012 to 2025. *International Journal of Sports Science & Coaching*.
- [9] MinusKelvin. (2019). Cold Clear: Tetris Bot. GitHub. <https://github.com/MinusKelvin/cold-clear>
- [10] n3koasakura. (2024). Tetr.io Top Players Replays (Tetris). Kaggle Dataset. <https://www.kaggle.com/datasets/n3koasakura/tetr-io-top-players-replays>
- [11] Stevens, M. (2016). Playing Tetris with Deep Reinforcement Learning. Stanford CS231N Reports.
- [12] Thiery, C. & Scherrer, B. (2009). Improvements on Learning Tetris with Cross Entropy. *International Computer Games Association Journal*, 32(1), 23-33.



## 부록 A: 시각 자료 기획 가이드

### A.1 Fig. 1 - 시스템 아키텍처 흐름도

4계층(Data Acquisition -> Statistical Analysis -> AI Intelligence -> Feedback Generation) 구조. 각 계층에 소속 모듈과 데이터 흐름 표시. 본 논문 개요 작성 시 생성된 SVG 다이어그램을 기반으로 고해상도 벡터 이미지로 제작.

### A.2 Fig. 2 - Feature Importance 분석

(a) 수평 바 차트: 10개 특성의 중요도 순위. tspin\_rate(73.9%) 강조. (b) Confusion Matrix(5x5): speed/attack/tspin/defense/balanced. (c) ROC Curve: multi-class, one-vs-rest.

### A.3 Fig. 3 - 피드백 생성 파이프라인

입력(ttrm/API) -> 파싱 -> 집계 -> [ML 분류기 + 규칙 평가기 + 빌드 DB] -> 코칭 + 로드맵 + 매치업. 각 단계의 데이터 변환과 추상화 수준을 시각화.

### A.4 Table 3 - 빌드 패턴 x 티어 추천 매트릭스

16개 빌드(행) x 11개 티어(열)의 교차표. 해당 티어에서 추천되는 빌드에 체크 표시. 빌드별 공격 유형, 난이도, 출력 라인 병기.

### A.5 Fig. 4 - 사용자 대시보드 스크린샷

(a) 성장 그래프 탭: 6개 subplot (APM, PPS, VS, 가비지, T-Spin/Quad, Finesse). (b) 통계 요약 탭: 온라인/로컬 분기 표시. (c) AI 코칭 탭: 8개 섹션 피드백 텍스트. (d) 훈련 로드맵 탭: 20분 루틴.